

Efficient and Effective Biclique Counting with Local Differential Privacy

Anonymous Author(s)

ABSTRACT

Counting (p, q) -bicliques in bipartite graphs is essential for uncovering dense substructures and higher-order patterns. However, releasing biclique counts can reveal private connections of users. To address this, we study (p, q) -biclique counting under edge local differential privacy, where each vertex protects its one-hop neighbor relationships from an untrusted data curator. The naive algorithm applies randomized responses to the adjacency matrix and produces an overly dense graph where (p, q) -biclique structures are disrupted, resulting in highly biased estimates. To produce unbiased estimates, we propose a one-round algorithm that enumerates all motifs with p upper and q lower vertices, which leverages motif transformation probabilities to correct for perturbation-induced bias. To improve data utility and reduce the impact of Laplace noise, we propose a novel Multi-round Common Neighbor (MRCN) algorithm. MRCN estimates the number of common neighbors for each p -tuple (or q -tuple) and applies moment-based correction to recover unbiased (p, q) -biclique counts. Notably, MRCN avoids explicit enumeration of all (p, q) -motifs, offering better scalability for general p and q . We further boost performance with variance-reduction techniques such as multi-center optimization and refined noisy graph construction, and enhance scalability through layer-based pruning and vertex sampling. Extensive experiments on real-world datasets confirm the effectiveness and efficiency of our approach under edge LDP.

ACM Reference Format:

Anonymous Author(s). 2025. Efficient and Effective Biclique Counting with Local Differential Privacy. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Bipartite graphs are a natural fit for modeling relationships between two distinct sets of entities, such as user-item interactions in e-commerce [20], people-location networks in contact tracing [1], and user-page networks in social media analysis [24]. (p, q) -biclique counting has been widely studied [27, 39, 41] since it captures higher-order cohesive structures with p upper and q lower vertices. This enables more fine-grained pattern detection in applications such as fraud detection [35, 36], dense subgraph mining, and link prediction [12]. However, releasing exact (p, q) -biclique counts

can compromise privacy by revealing sensitive relationships, such as a user's interaction with a specific item. For example, consider

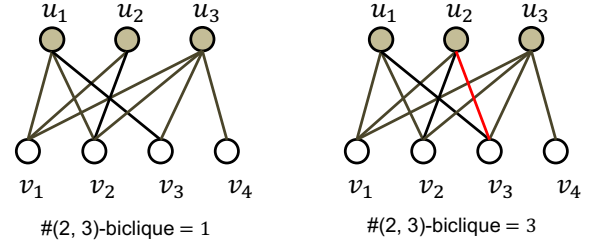


Figure 1: An example of how $(2, 3)$ -biclique counts reveal private connections.

the user-item networks in Fig. 1, where each edge indicates a user's purchase of an item. In the bipartite graph on the left, there are 3 users and 4 items, and one $(2, 3)$ -biclique formed by users (u_1, u_3) and items (v_1, v_2, v_3) . When user u_2 additionally purchases item v_3 , edge (u_2, v_3) is added, resulting in the graph on the right. Suppose users u_1 and u_3 collude to infer u_2 's purchases. If the $(2, 3)$ -biclique counts of the two graphs are released, the colluding users can observe a jump (e.g., from 1 to 3) and deduce that u_2 purchased v_3 , leading to a potential privacy breach. Thus, it is vital to estimate the number of (p, q) -bicliques of a bipartite graph in a privacy-preserving manner.

Motivated by these observations, we study the problem of privacy-preserving (p, q) -biclique counting on bipartite graphs. Specifically, given a bipartite graph G , we aim to compute the (p, q) -biclique count under *edge local differential privacy* (edge LDP) [25, 40, 43]. A random algorithm satisfies edge LDP if, for any two vertices whose neighbors differ by one edge (u, v) , the probabilities of observing the algorithmic outputs from these two vertices are indistinguishable within a factor of e^ϵ . This provides deniability for the existence of the edge (u, v) . Here, ϵ represents the privacy budget, which controls the tradeoff between privacy and data utility, allowing clients to adjust the level of privacy protection. Edge LDP ensures robust privacy by requiring each vertex to perturb its local data before transmission to the data curator, eliminating the need for a trusted curator, unlike *central differential privacy* (CDP) [2, 4, 7, 8, 18, 23]. Compared to CDP, where a compromised curator with access to the entire graph endangers all users' privacy [13, 17], edge LDP provides a more resilient privacy mechanism.

Existing methods. Motif counting under edge LDP typically follows two computational paradigms: (1) one-round and (2) multi-round [9, 11, 13, 15]. In one-round algorithms, the entire privacy budget is used to construct a noisy graph by applying randomized response to the adjacency matrix of the original graph G , where each entry is independently flipped with a fixed probability. Motif counts,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA

such as butterfly and triangle counts, are then estimated from this noisy graph. While no one-round methods have been studied for general (p, q) -bicliques, He et al. [9] propose a one-round algorithm for butterfly counting, a special case with $p = q = 2$. In multi-round algorithms, each vertex downloads the noisy graph from earlier rounds, estimates the per-vertex motif counts locally, and reports these estimators with calibrated Laplace noise. While (p, q) -biclique counting under this classic framework is briefly discussed in the context of k -star LDP [31], it violates edge LDP by transmitting uncalibrated local counts. Moreover, relying on per-vertex motif counts within the multi-round framework leads to excessive noise, especially for complex structures like (p, q) -bicliques.

Challenges. In this paper, we aim to design accurate and efficient algorithms for counting (p, q) -bicliques under edge LDP and face the following challenges.

- *Challenge 1:* Given a bipartite graph G , randomized responses produce a much denser noisy graph G' by independently flipping edges [25], which often destroys complex structures like (p, q) -bicliques. As a result, a naive algorithm that directly counts (p, q) -bicliques on G' suffers from significant bias. It is a challenge to derive unbiased estimates for (p, q) -bicliques under edge LDP.
- *Challenge 2:* Under randomized responses, each (p, q) -biclique may be transformed into any other subgraph with p upper and q lower vertices due to edge flipping. Recovering the original (p, q) -biclique count thus often requires enumerating all such (p, q) -vertex bipartite subgraphs, which is computationally expensive and quickly becomes infeasible for arbitrary p and q .

Our Approaches. To address *Challenge 1*, we extend the one-round algorithm for butterfly counting under edge LDP [9] and propose a OneR algorithm that provides unbiased (p, q) -biclique estimates based on the noisy graph. Our proposed OneR algorithm carefully analyzes how motifs with p upper and q lower vertices transform into each other with different probabilities under randomized response. Specifically, OneR constructs a noisy graph G'_ϵ and counts the number of motifs with p upper vertices, q lower vertices and i edges, for each i in $0 \leq i \leq pq$. Using these counts, it solves a linear system based on the transformation matrix of motif probabilities to derive a closed-form, unbiased estimator for the (p, q) -biclique count. This corrects for bias by accounting for expected motif transformations due to randomized response. In the meantime, as OneR depends on roughly $p \times q$ independent Bernoulli variables to estimate each (p, q) -biclique, its performance can be further improved, especially on sparse graphs. Also, OneR requires enumerating all (p, q) -vertex motifs, which becomes computationally challenging for general values of p and q .

To address *Challenge 2*, we propose a novel multi-round algorithm, MRCN, for (p, q) -biclique counting that avoids enumerating all subgraphs with p upper and q lower vertices. Unlike classic multi-round approaches, MRCN does not rely on per-vertex biclique counts and thus avoids injecting substantial Laplace noise into the estimates. Instead, MRCN estimates the number of common neighbors for each p -tuple in the upper layer or each q -tuple in the lower layer of G , leveraging higher moments of these estimators to derive unbiased per-tuple biclique counts. Specifically, MRCN estimates the vertex degrees in the first round and builds a noisy graph via randomized responses in the second round. In the third round, for each p -tuple X , the vertex with the smallest estimated degree is

chosen as the central vertex. We estimate how many of its neighbors are also connected to the remaining vertices in X , and add Laplace noise calibrated to the sensitivity using budget ϵ_2 . Then, MRCN estimates the variance of each common neighbor estimator to compute its higher moments. These are used to derive unbiased per-tuple biclique estimates, which are aggregated to obtain the total (p, q) -biclique count. MRCN fully leverages the multi-round setting, allowing the data curator to estimate common neighbors by aggregating local neighborhood information. This design also avoids the need to handle the intractable global sensitivity of bicliques and quasi-bicliques, a key limitation of classic multi-round frameworks. Moreover, MRCN is more efficient than OneR, reducing the dominant computation from enumerating all (p, q) -motifs to processing p -tuples in $U(G)$ or q -tuples in $L(G)$.

To further improve the data utility of MRCN, we propose the MRCN++ algorithm with two variance-reduction techniques: *multi-center optimization* and *refined noisy graph construction*. The *multi-center* method selects multiple upper-layer vertices to build independent estimators for per-tuple common neighbors and averages them, which reduces noise from individual vertices. The *refined noisy graph construction* leverages the parallel and post-processing properties of edge LDP to generate two independent noisy graphs under the same privacy budget by applying randomized response separately to the upper and lower layers. This reduces the variance of each adjacency estimate $\mathcal{A}[i, j]$ and improves the accuracy of (p, q) -biclique counting. Additionally, we propose a *layer-based optimization* that chooses between enumerating p -tuples in $U(G)$ or q -tuples in $L(G)$ based on estimated computational cost, and a *vertex tuple sampling* technique that limits the number of tuples enumerated to improve scalability while maintaining unbiasedness.

Contributions. Here, we outline our main contributions.

- We study the (p, q) -biclique counting problem under edge LDP. We propose the OneR algorithm that corrects for motif transformations caused by randomized response. OneR produces unbiased (p, q) -biclique estimation and works well on dense bipartite graphs.
- We propose a novel multi-round common-neighbor-based algorithm, MRCN, which estimates the number of common neighbors for each p -tuple in the upper layer (or q -tuple in the lower layer). By correcting higher-order moments of these estimators, MRCN yields more accurate and unbiased estimates of the (p, q) -biclique count.
- We propose the MRCN++ algorithm with two variance-reduction techniques: multi-center optimization and refined noisy graph construction. Also, we employ layer-based optimization and vertex sampling to improve efficiency.
- We conduct extensive experiments on 13 real-world bipartite graphs to evaluate the effectiveness and efficiency of our algorithms. Results show that OneR, MRCN, and MRCN++ consistently outperform the Naïve algorithm in data utility. Notably, MRCN achieves mean relative errors up to three orders of magnitude lower than Naïve. Also, MRCN++ with the proposed optimizations consistently outperforms MRCN in most parameter settings.

Organization. The rest of the paper is organized as follows. Section 2 defines the problem and presents the Naïve algorithm. Section 3 presents the OneR algorithm. Section 4 presents the MRCN algorithm and the MRCN++ algorithm with optimizations. Section 5 reports and analyzes the experimental results. Section 6 reviews the related works, and Section 7 concludes the paper.

2 PROBLEM DEFINITION

Table 1: Summary of Notations

Notation	Definition
G	a bipartite graph
$\mathcal{A}$	the adjacency matrix
$\mathcal{A}_u$	the neighbor list of the vertex u
$N(u, G)$	the neighbors of u in G
$\deg(u, G)$	the degree of u in G
ϵ	a privacy budget
G'	a noisy bipartite graph
$\omega(X)$	the number of common neighbors among X
Δg	the global sensitivity of a function g
$\mathcal{K}_{p,q}$	number of (p, q) -bicliques in graph G
$\mathcal{K}_{p,q}(X)$	(p, q) -biclique count containing the vertex set X

In this section, we first present important definitions and notations about (p, q) -biclique and differential privacy on bipartite graphs. Then, we present the problem statement.

2.1 (p, q) -biclique Counting on Bipartite Graphs

We consider an unweighted bipartite graph $G(V = (U, L), E)$. $V = U \cup L$ denotes the set of vertices, where U and L represent the upper and lower layers, respectively. The vertices in U and L are called upper vertices and lower vertices, respectively. $E \subseteq U \times L$ denotes the set of edges. Let $n_1 = |U(G)|$, $n_2 = |L(G)|$, and $m = |E(G)|$ denote the number of upper vertices, lower vertices, and edges, respectively. The adjacency matrix $\mathcal{A}$ for G is of dimensions $n \times n$, where $\mathcal{A}[u, v] = 1$ if there exists an edge between the vertices u and v and 0 otherwise. The u -th row of $\mathcal{A}$ (including both “1” and “0”) is the *neighbor list* of u , denoted by $\mathcal{A}_u$. In addition, the set of neighbors of a vertex u in G is denoted by $N(u, G)$, and its degree is denoted by $\deg(u, G) = |N(u, G)|$. We denote by $d_{\max}^1$ and $d_{\max}^2$ the maximum degrees of vertices in the upper set $U(G)$ and lower set $L(G)$, respectively.

In this paper, we are interested in the number of bicliques on bipartite graphs. Like cliques on general graphs, bicliques serve as compact representations of highly connected subgraphs on bipartite graphs. We consider the bicliques with p upper vertices and q lower vertices, as defined below.

DEFINITION 1. (p, q) -biclique. Given a bipartite graph G and two integers p and q , a (p, q) -biclique is a complete subgraph of G with p upper vertices and q lower vertices.

2.2 Differential privacy on bipartite graphs

Classic DP assumes a central model, where a central data curator manages a dataset D [5]. Two datasets D and D' are considered neighboring datasets if they differ by one data record. For any neighboring datasets, DP ensures that the outputs of a query function on these datasets are hard to distinguish. When DP under the central model is applied to graphs to protect edge privacy, the neighboring datasets are two graphs that differ by one edge. While this central model offers better data utility, the assumption that the data curator storing the entire graph can be trusted is often unattainable. Thus, we adopt the widely-used ϵ -edge LDP, which allows each

vertex to perturb its data locally before sending it to the data curator [26, 40, 43]. Under edge LDP, the neighboring datasets are two neighbor lists that differ by one bit.

DEFINITION 2. ϵ -edge local differential privacy. Let G be a bipartite graph and $\epsilon > 0$. For each vertex $u \in V(G)$, let R_u with domain $\{0, 1\}^n$ be a randomized algorithm of vertex u . R_u provides ϵ -edge LDP if for any two neighbor lists $\mathcal{A}_u$ and $\mathcal{A}'_u$ that differ in one bit and any $S \subseteq \text{Range}(R_i)$:

$$\Pr(R_u(\mathcal{A}_u) \in S) \leq e^\epsilon \Pr(R_u(\mathcal{A}'_u) \in S)$$

Here, ϵ is referred to as the privacy budget.

Note that ϵ -edge LDP ensures that if two vertices have neighbor lists that differ by one bit, they cannot be reliably distinguished based on the outputs from the randomized algorithms. Consider t edge LDP algorithms $Z_1, Z_2, \dots, Z_t$, whose privacy budgets are denoted by $\epsilon_1, \epsilon_2, \dots, \epsilon_t$, respectively. We have the following property.

THEOREM 1. Sequential Composition [17]. The composite algorithm obtained by sequentially applying $Z_1, Z_2, \dots, Z_t$ on the bipartite graph G provides $\sum_{i=1}^t \epsilon_i$ -differential privacy.

Randomized responses. The easiest way to provide ϵ -edge LDP is via randomized responses (hereafter denoted by RR), which was originally proposed as a survey technique to allow confidential answers to sensitive issues (e.g. criminal or sexual activities) [37]. Essentially, the participants are asked to answer questions truthfully with probability. This has been applied to graphs to satisfy edge local differential privacy [13, 25]. Specifically, given a privacy budget ϵ , each entry $x \in \{0, 1\}$ of a neighbor list is perturbed with probability $\frac{1}{1+e^\epsilon}$:

$$RR(x) = \begin{cases} 1 - x & \text{with probability } \frac{1}{1+e^\epsilon} \\ x & \text{with probability } \frac{e^\epsilon}{1+e^\epsilon} \end{cases}$$

Existing works on bipartite graphs [9, 10] restrict randomized response to the upper-right block. This avoids conflicts in the noisy graph where both $\mathcal{A}'[i, j]$ (perturbed by i) and $\mathcal{A}'[j, i]$ (perturbed by j) might be inconsistent. Specifically, for each $u \in U(G)$, only entries in $\mathcal{A}_u \cap L(G)$ corresponding to the possible neighbors of u are perturbed. We denote the noisy graph w.r.t. a privacy budget ϵ as G'_ϵ .

Calibrating noise to global sensitivity. To ensure ϵ -edge LDP, any data transmitted from a vertex to the data curator requires the addition of noise. The Laplace Mechanism [5] is often used to draw noise proportional to the global sensitivity of the data. We formally define global sensitivity here.

DEFINITION 3. Global sensitivity. Consider a bipartite graph G . Let $\mathcal{A}_u$ be the neighbor list of vertex u . The global sensitivity of a function $f : \mathcal{A}_u \rightarrow \mathbb{R}$ is:

$$\Delta f = \max_{u, u' \in V(G)} |f(\mathcal{A}_u) - f(\mathcal{A}'_u)|$$

Here, $\mathcal{A}'_u$ is a neighbor list that differs from $\mathcal{A}_u$ in at most one entry. Δf represents the global sensitivity of f .

DEFINITION 4. The Laplace Mechanism. Given a privacy budget ϵ and any function f , the Laplace mechanism is defined as:

$$\tilde{f} = f + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right)$$

where $\tilde{f}$ is the noisy version of f and $\text{Lap}(\cdot)$ is the probability density function of the Laplace distribution.

Applying the Laplacian mechanism, we allow the vertices to send local graph statistics with calibrated noise to the data curator while satisfying ϵ -edge LDP.

Problem Statement. Given a bipartite graph G , two integers p and q , a privacy budget ϵ , we aim to accurately and efficiently estimate the number of (p, q) -bicliques in G while satisfying ϵ -edge LDP.

Algorithm 1: Naive

Input: G : a bipartite graph; p, q : integers; ϵ : a privacy budget;
Output: an estimate of the number of (p, q) -bicliques
// vertex side:
1 **foreach** $i \in U(G)$ **do**
2 **foreach** $j \in L(G)$ **do**
3 perturb $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^\epsilon} \\ \mathcal{A}[i, j] & \text{w.p. } \frac{e^\epsilon}{1+e^\epsilon} \end{cases}$
4 send noisy edges to the data curator;
// curator side:
5 $G'_\epsilon \leftarrow$ the noisy graph constructed from $\mathcal{A}'[i, j]$;
6 **return** the (p, q) -biclique count on G'_ϵ ;

A naive approach. Since the randomized responses preserve ϵ -edge LDP, a naive solution returns the (p, q) -biclique count of the noisy graph from the randomized responses, as detailed in Algorithm 1. Here, we use $\mathcal{A}'$ to indicate the adjacency matrix for the noisy graph G' . In Lines 1-4, each upper vertex perturbs its neighbor list and sends it to the data curator. Specifically, given a privacy budget $\epsilon > 0$, each entry $\mathcal{A}[i, j]$ with i and j being vertices on different layers is flipped with a probability $\frac{1}{1+e^\epsilon}$. In Lines 5-6, the data curator constructs a noisy graph G'_ϵ and returns the number of butterflies on G'_ϵ obtained by calling the state-of-the-art biclique counting algorithm [34]. Since most neighbor lists are sparse (with more “0” than “1”), flipping them with a fixed probability $\frac{1}{1+e^\epsilon}$ makes G'_ϵ much denser than G , which generally leads to over-counting.

Computational time complexity. We analyze the computational time cost and communication costs of Naive. During the randomized response stage, the time cost is $O\left(n_1 n_2 \cdot \frac{1}{1+e^\epsilon}\right)$ due to the edge flipping probability of $\frac{1}{1+e^\epsilon}$. In the counting stage, the dominant cost arises from (p, q) -biclique counting on the noisy graph G'_ϵ , which requires $O\left(\min(n_1^p, n_2^q)\right)$ time in the worst-case scenario. This is because the state-of-the-art (p, q) -biclique counting algorithm [39, 41] runs in $O\left(\min(n_1^p, n_2^q)\right)$ time. Thus, the overall time costs of Naive are $O\left(\frac{n_1 n_2}{1+e^\epsilon} + \min(n_1^p, n_2^q)\right)$.

Per-vertex Communication costs. The communication costs of Naive occur during randomized responses, when each vertex uploads the noisy edges to the data curator. Each noisy edge incurs a cost of $O(\log(n_1 n_2))$, since each edge can be represented by two vertex IDs [14]. The expected number of noisy edges sent by a vertex u is $\deg(u)(1-p_r) + (n_2 - \deg(u))p_r = O\left(\frac{n_2}{1+e^\epsilon}\right)$. Thus, the per-vertex communication cost is $O\left(\frac{n_2}{1+e^\epsilon} \cdot \log(n_1 n_2)\right) = O\left(\frac{n_2 \log(n_1 n_2)}{1+e^\epsilon}\right)$.

3 ONE ROUND APPROACH

The Naive algorithm in Section 2 leads to significant bias because the noisy graph is much denser than the original graph. Recently, [9] proposes a multi-round algorithm and a non-interactive algorithm for butterfly counting under edge LDP, where the non-interactive approach achieves superior data utility as it prevents injecting substantial noise from the high global sensitivity in the classic multiple-round computing framework. The non-interactive algorithm relies on randomized responses to build a noisy graph and carefully model the transformation probabilities of all motifs with two upper and lower vertices to get unbiased butterfly count estimates. Since the butterfly is a special case of (p, q) -biclique, we extend this idea to (p, q) -biclique and propose a one-round algorithm that produces an unbiased estimate of (p, q) -biclique counts. To begin with, we revisit how motifs transform into each other with different probabilities due to randomized responses.

3.1 Motif transformation during randomized responses

To recover the (p, q) -biclique count on G , we need to investigate how the motifs with p upper vertices and q lower vertices transform into each other with different probabilities. Here, we define the transformation probability.

DEFINITION 5. Transformation probability [9]. Given a privacy budget ϵ and a bipartite graph G , let G'_ϵ be a noisy graph from applying randomized response to G . Let $\mathcal{M}$ be a motif in G and $\mathcal{N}$ be a motif in G'_ϵ . The probability that the motif in G'_ϵ induced by the vertices in $\mathcal{M}$ is $\mathcal{N}$ is called the transformation probability from $\mathcal{M}$ to $\mathcal{N}$, i.e.,

$$TP(\mathcal{M}, \mathcal{N}) = P(G'[V] \cong \mathcal{M} \mid G[V] \cong \mathcal{N})$$

Here $\mathcal{V}$ denotes a vertex set in $V(G)$. The symbol $\cong$ denotes graph isomorphism between two graphs.

As shown in Fig. 2, a $(2, 3)$ -biclique can transform into 16 distinct types of motifs with different probabilities. For instance, the transformation probability of a $(2, 3)$ -biclique to itself is $(1-p)^6$ as it requires each of the 6 edges to stay un-flipped during randomized responses ($p = 1/(1+e^\epsilon)$). In what follows, we consider the motifs with the same number of edges together and use $\mathcal{B}_i$ to denote the motifs with i edges ($0 \leq i \leq p \times q$). The numbers of $\mathcal{B}_i$ on G and G' are denoted by $n_{\mathcal{B}_i}$ and $n'_{\mathcal{B}_i}$, respectively. Then, we have the following equation.

$$\mathbb{E}(n'_{\mathcal{B}_1}, n'_{\mathcal{B}_2}, \dots, n'_{\mathcal{B}_{p \times q}}) = (n_{\mathcal{B}_1}, n_{\mathcal{B}_2}, \dots, n_{\mathcal{B}_{p \times q}}) \times \mathcal{T}$$

Here $\mathcal{T}$ is the transition matrix where $\mathcal{T}[i, j] = TP(\mathcal{B}_i, \mathcal{B}_j)$. In this way, the transition matrix $\mathcal{T}$ stores the transformation probabilities from $\mathcal{B}_i$ to $\mathcal{B}_j$, $\forall i, j$. Based on the above equation, we obtain the following linear system for (p, q) -biclique counting.

$$(\hat{n}_{\mathcal{B}_0}, \hat{n}_{\mathcal{B}_1}, \dots, \hat{n}_{\mathcal{B}_{p \times q}}) = (n'_{\mathcal{B}_0}, n'_{\mathcal{B}_1}, \dots, n'_{\mathcal{B}_{p \times q}}) \times \mathcal{T}^{-1} \quad (1)$$

Since $C_{p,q}(G) = n_{\mathcal{B}_{pq}}$, we solve for $\hat{n}_{\mathcal{B}_{pq}}$ as the unbiased estimate for $C_{p,q}(G)$ in the theorem below.

THEOREM 2. Given a bipartite graph G , a privacy budget ϵ , let G'_ϵ be the noisy graph after applying randomized response to G w.r.t. ϵ .

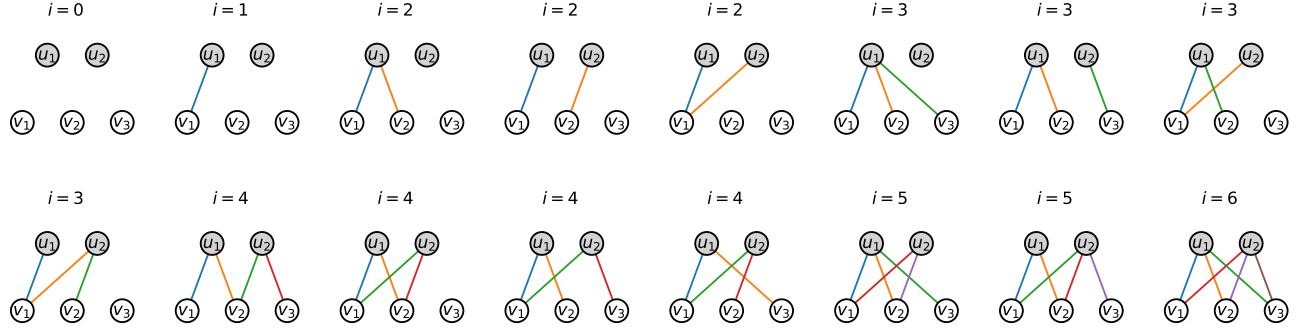


Figure 2: All possible motifs with 2 upper and 3 lower vertices. Each motif is annotated with its edge count i .

Let $\mu = e^\varepsilon$. We have

$$\mathbb{E}(\hat{n}_{\mathcal{B}_{pq}}) = C_{p,q}(G)$$

where $\hat{n}_{\mathcal{B}_{pq}} = \frac{\sum_{i=0}^{p \times q} (-1)^i \mu^i n'_{\mathcal{B}_i}}{(1-\mu)^{p \times q}}$.

Algorithm 2: One-Round $(2, q)$ -Biclique Counting under Edge LDP

Input: G : a bipartite graph with vertex sets U and L ; q : target size on L side; ε : privacy budget;
Output: an estimate of the number of $(2, q)$ -bicliques
// vertex side: apply randomized response

```

1 foreach  $i \in U$  do
2   foreach  $j \in L$  do
3     perturb edge presence:
4      $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^\varepsilon} \\ \mathcal{A}[i, j], & \text{w.p. } \frac{e^\varepsilon}{1+e^\varepsilon} \end{cases}$ 
5 send noisy edges  $\mathcal{A}'$  to the curator;
// curator side: counting motifs with two upper and  $q$  lower vertices
6 Construct noisy graph  $G'_\varepsilon$  from  $\mathcal{A}'$ ;
7 Initialize counters  $m_0, m_1, \dots, m_{2K} \leftarrow 0$ ;
8 foreach pair  $(u_1, u_2) \in U(G')$  do
9   Compute  $s_0, s_1, s_2$  based on neighborhoods in  $G'_\varepsilon$ ;
10  for  $c = 0$  to  $q$  do
11    for  $b = 0$  to  $q - c$  do
12       $a \leftarrow q - b - c$ ;
13       $i \leftarrow 2c + b$ ;
14       $m_i \leftarrow m_i + \binom{s_0}{a} \cdot \binom{s_1}{b} \cdot \binom{s_2}{c}$ ;
// estimate correction using inverse transform
15 Let  $\mu \leftarrow e^\varepsilon$ ;
16  $\hat{n}_{\mathcal{B}_{2q}} \leftarrow \frac{1}{(1-\mu)^{2q}} \cdot \sum_{i=0}^{2q} (-\mu)^i \cdot m_i$ ;
17 return  $\hat{n}_{\mathcal{B}_{2q}}$ ;

```

3.2 A one-round algorithm

Based on Theorem 2, we propose a one-round algorithm that returns an unbiased estimate of the number of (p, q) -bicliques. The algorithm first constructs the noisy graph G'_ε using the full privacy

budget ε . It then counts the number of motifs with p upper and q lower vertices in G'_ε , and applies the formula in Theorem 2 to compute the unbiased estimate.

In practice, no general efficient algorithm exists for counting motifs with p upper and q lower vertices for arbitrary (p, q) values. Specialized counting strategies must be developed for each case. In the following, we present the one-round algorithm for the specific case of $p = 2$ as an example.

An example of the one-round algorithm when $p = 2$. Algorithm 2 presents a specialized implementation of the one-round algorithm for the case where $p = 2$ and the lower side has size q . In the counting phase (Lines 8–14), Algorithm 2 iterates over each upper vertex pair (u_1, u_2) and classifies lower vertices into s_2, s_1 , and s_0 , denoting the counts of vertices connected to both, exactly one, or neither vertex in the noisy graph G'_ε . For each triple (a, b, c) with $a + b + c = q$, the algorithm counts the number of $(2, q)$ motifs where a lower vertices connect to neither u_1 nor u_2 , b to exactly one, and c to both. The count for motifs with $i = 2c + b$ edges is incremented by $\binom{s_0}{a} \cdot \binom{s_1}{b} \cdot \binom{s_2}{c}$. In this way, Algorithm 2 efficiently counts the number of motifs with two upper vertices and q lower vertices in G'_ε . Finally, Lines 15–16 apply the closed-form formula from Theorem 2 to recover an unbiased estimate of the number of $(2, q)$ -bicliques in the original graph G .

Computational time complexity. We analyze the computational time cost of OneR. The time costs incurred during randomized response are the same as Naive, which is $O\left(\frac{n_1 n_2}{1+e^\varepsilon}\right)$. Unlike Naive, OneR needs to count all motifs with p upper vertices and q lower vertices on G'_ε , which takes $O\left(\binom{n_1}{p} \cdot \binom{n_2}{q} \cdot pq\right)$. $\binom{n_1}{p}$ and $\binom{n_2}{q}$ denote the number of ways to choose p and q vertices from U and L , and pq accounts for the time to examine the number of edges in each motif. Thus, the time complexity of OneR is $O\left(\frac{n_1 n_2}{1+e^\varepsilon} + \binom{n_1}{p} \cdot \binom{n_2}{q} \cdot pq\right)$. When $p = 2$, our implementation (Algorithm 2) iterates over all upper vertex pairs and computes motif counts via neighborhood intersections, which takes $O(n_1^2 \cdot q^2)$ time in the counting phase. Overall, the time complexity is

$$O\left(\frac{n_1 n_2}{1+e^\varepsilon} + n_1^2 q^2\right)$$

when $p = 2$.

Per-vertex Communication costs. The communication costs of OneR occur during randomized responses, just like Naïve, which incurs a per-vertex communication cost of

$$O\left(\frac{n_2 \log(n_1 n_2)}{1 + e^\epsilon}\right).$$

4 A NOVEL MULTI-ROUND FRAMEWORK

The one-round algorithm for (p, q) -biclique counting alleviates the issue of dense, noisy graphs by modeling motif transformation probabilities under randomized responses and returns unbiased estimates. A key issue with OneR is that each edge is flipped independently, so each (p, q) -biclique depends on roughly $p \times q$ independent Bernoulli variables, resulting in high variance, especially on sparse graphs. Also, it requires a tailored algorithm for each (p, q) to count how many subgraphs with p upper and q lower vertices have exactly i edges, for each $i \in [0, pq]$, which is generally challenging, especially when $p > 2$. Thus, we divert our attention to the multi-round algorithms, which have been widely applied to triangle counting [11, 13, 14]. To perform (p, q) -biclique counting under the classic multi-round framework, we allocate part of the privacy budget to constructing a noisy graph via randomized response, and have each vertex estimate the number of (p, q) -bicliques and quasi- (p, q) -bicliques in its local graph formed by overlapping its neighborhood with the noisy graph. To report the local motif counts to the central data curator, we need to inject substantial noise into them to satisfy edge LDP via the Laplace mechanism because the global sensitivities of noisy (p, q) -bicliques are very large due to their complex structure.

To tackle high global sensitivity in noisy (p, q) -biclique counts, we propose a novel **Multi-Round Common Neighbor-based algorithm** (MRCN) for unbiased (p, q) -biclique counting under edge LDP. (1) We begin by estimating the degree of each vertex using the Laplace mechanism. (2) We construct a noisy graph via randomized response. (3) For each set of p vertices, we estimate the number of their common neighbors, along with the higher-order moments of this estimator. These estimates are then used to construct an unbiased estimator for the number of (p, q) -bicliques. For ease of presentation, we first present the algorithm for the case $p = 2$, and then discuss how to extend it to larger values of p .

4.1 A working example for $p = 2$

We begin with the special case of $p = 2$. To count $(2, q)$ -bicliques under edge LDP, we consider all vertex pairs in $U(G)$, estimate their number of common neighbors, and derive the $(2, q)$ -biclique count for each pair. We build on a recent work in common neighbor estimation under edge LDP [10] and discuss how to obtain unbiased per-vertex-pair $(2, q)$ -biclique count estimators.

An unbiased estimator for common neighbors per vertex pair. The common neighbor estimator builds on the unbiased estimator for each adjacency entry $\mathcal{A}[i, j]$ in a bipartite graph G . Given privacy budget ϵ_1 , we construct a noisy graph G'_{ϵ_1} by applying randomized response to each edge, flipping its presence with probability $p_r = \frac{1}{1+e^{\epsilon_1}}$. Each noisy entry $\mathcal{A}'[i, j]$ then follows a Bernoulli distribution with parameter $1 - p_r$ if $\mathcal{A}[i, j] = 1$, and p_r if $\mathcal{A}[i, j] = 0$. Based on this, we have the following equations which

link the expected value of $\mathcal{A}[i, j]'$ and $\mathcal{A}[i, j]$:

$$\mathbb{E}(\mathcal{A}'[i, j]) = \begin{cases} p_r, & \text{if } \mathcal{A}[i, j] = 0 \\ 1 - p_r & \text{if } \mathcal{A}[i, j] = 1 \end{cases}$$

This can be rearranged into:

$$E(\phi(i, j)) = \mathcal{A}[i, j].$$

Here, $\phi(i, j) = \frac{\mathcal{A}'[i, j] - p_r}{1 - 2p_r}$, which is an unbiased estimator of $\mathcal{A}[i, j]$. The variance of $\phi(i, j)$ is given by

$$\text{Var}(\phi(i, j)) = \text{Var}\left(\frac{\mathcal{A}'[i, j] - p_r}{1 - 2p_r}\right) = \frac{p_r(1 - p_r)}{(1 - 2p_r)^2}$$

Assume the noisy graph G_{ϵ_1} has been constructed and downloaded by each vertex. For a vertex pair (u, w) with $u, w \in U(G)$, the number of common neighbors can be locally estimated by u using the unbiased estimator $\phi(v, w)$ for each $v \in N(u, G)$. Specifically, $\sum_{v \in N(u, G)} \phi(v, w)$ provides an unbiased estimate of the common neighbors between u and w . To report this to the data curator, Laplace noise must be added based on the global sensitivity of the estimator, which is $\frac{1 - p_r}{1 - 2p_r}$. Thus, the unbiased estimator at the curator becomes

$$f_u = \sum_{v \in N(u, G)} \phi(v, w) + \text{Lap}\left(\frac{1 - p_r}{(1 - 2p_r)\epsilon_2}\right).$$

The variance of f_u arises from two sources: the sum $\sum_{v \in N(u, G)} \phi(v, w)$ and the Laplace noise. The variance contributed by the first term is $\frac{p_r(1 - p_r)}{(1 - 2p_r)^2} \deg(u)$, since each $\phi(v, w)$ is independent and there is no covariance. The variance of the Laplace noise is

$$\text{Var}\left(\text{Lap}\left(\frac{1 - p_r}{(1 - 2p_r)\epsilon_2}\right)\right) = 2 \left(\frac{1 - p_r}{(1 - 2p_r)\epsilon_2}\right)^2 = \frac{2(1 - p_r)^2}{(1 - 2p_r)^2 \epsilon_2^2}.$$

Thus, the total variance of f_u is

$$\text{Var}(f_u) = \frac{p_r(1 - p_r)}{(1 - 2p_r)^2} \deg(u) + \frac{2(1 - p_r)^2}{(1 - 2p_r)^2 \epsilon_2^2}.$$

Estimating $\mathbb{K}_{2,q}(u, w)$ for the vertex pair (u, w) . Let $\omega(u, w)$ denote the true number of common neighbors between u and w , and let f_u be the unbiased estimate of $\omega(u, w)$ obtained from u 's local neighborhood. To estimate the number of $(2, q)$ -bicliques involving u and w , denoted by $\mathbb{K}_{2,q}(u, w)$, we use the combinatorial formula: $\mathbb{K}_{2,q}(u, w) = \binom{\omega(u, w)}{q}$. Using (f_u) as a plug-in estimator introduces bias, as the expectation is not preserved under nonlinear transformations such as polynomials. To address this, we introduce a *moment correction technique* that: (1) expresses $\binom{\omega(u, w)}{q}$ as a polynomial in $\omega(u, w)$, (2) derives unbiased estimators for each monomial ω^k using the higher moments of f_u , and (3) combines them to construct an unbiased estimator for $\binom{\omega(u, w)}{q}$. We first illustrate the method for $q = 2$ and then extend it to handle general $q > 2$.

(1) When $p = 2$ and $q = 2$, we count the number of butterflies in G . For a vertex pair (u, w) , the butterfly count is $\mathbb{K}_{2,2}(u, w) = \binom{\omega(u, w)}{2} = \frac{\omega(u, w)^2 - \omega(u, w)}{2}$. Given the estimator f_u for $\omega(u, w)$, we aim to estimate $\omega(u, w)^2$ by leveraging the variance: $\text{Var}(f_u) =$

$\mathbb{E}(f_u^2) - \mathbb{E}(f_u)^2$. Thus, we have $\mathbb{E}(f_u^2 - \text{Var}(f_u)) = \omega(u, w)^2$. Based on this, we obtain the following unbiased estimator for $\mathcal{K}_{2,2}(u, w)$:

$$\hat{\mathcal{K}}_{2,2}(u, w) = \frac{f_u^2 - f_u - \text{Var}(f_u)}{2}.$$

Given the expression of $\text{Var}(f_u)$ obtained earlier, we can estimate it by replacing $\deg(u)$ with its unbiased estimate $\deg(u) = \deg(u, G) + \text{Lap}(1/\epsilon_0)$ from the first round. Based on the above discussion, we can obtain the following unbiased estimator for $\mathbb{K}_{2,q}$:

$$\hat{\mathbb{K}}_{2,2} = \sum_{u < w \in U(G)} \frac{f_u^2 - f_u - \hat{\text{Var}}(f_u)}{2}$$

(2) When $p = 2$ and $q > 2$, we apply the moment-based correction technique to estimate $\mathbb{K}_{2,q}(u, w)$. Specifically, we express the binomial coefficient as a degree- q polynomial:

$$\binom{\omega(u, w)}{q} = \sum_{k=0}^q a_k \cdot \omega(u, w)^k,$$

where the coefficients a_k depend only on q .

Each monomial ω^k is estimated using the k -th moment of f_u , corrected for the bias introduced by its variance. Assuming that f_u is approximately Gaussian with negligible higher-order cumulants, we compute ω^k for each k using the following recurrence [16]:

$$\widehat{\omega^k} = f_u^k - \sum_{i=1}^{\lfloor k/2 \rfloor} \binom{k}{2i} (2i-1)!! \cdot f_u^{k-2i} \cdot \hat{\sigma}^{2i},$$

where $\hat{\sigma}^2$ is the estimated variance of f_u , and $(2i-1)!!$ denotes the double factorial. These corrected moments are substituted into the binomial expansion to obtain an unbiased estimate of $\mathbb{K}_{2,q}(u, w)$.

Example (when $q = 3$): To estimate $\mathbb{K}_{2,3}(u, w) = \binom{\omega(u, w)}{3} = \frac{\omega^3 - 3\omega^2 + 2\omega}{6}$, we corrected moments derived from Equation : $\widehat{\omega} = f_u$, $\widehat{\omega^2} = f_u^2 - \hat{\sigma}^2$, and $\widehat{\omega^3} = f_u^3 - 3f_u \cdot \hat{\sigma}^2$. Substituting into the polynomial yields:

$$\hat{\mathbb{K}}_{2,3}(u, w) = \frac{f_u^3 - 3f_u \cdot \hat{\sigma}^2 - 3f_u^2 + 2f_u}{6}.$$

The MRCN algorithm when $p = 2$. We present the MRCN algorithm for $(2, q)$ -biclique counting under edge LDP in Algorithm 3. In *Phase 0*, each vertex adds Laplace noise to its degree. In *Phase 1*, a noisy bipartite graph is generated by independently perturbing each edge via randomized responses. In *Phase 2*, for each pair of upper-layer vertices (u, w) , the algorithm estimates the number of common neighbors using a local estimator from u 's perspective, denoted by f_u , and injects calibrated Laplace noise to preserve privacy. In *Phase 3*, MRCN computes the variance of f_u and applies the moment-based correction to construct an unbiased estimator for $\binom{\omega(u, w)}{q}$. These per-pair estimates are aggregated to yield the final unbiased estimate of the $(2, q)$ -biclique count.

4.2 The MRCN algorithm

In this subsection, we present the MRCN algorithm for general values of p , which relies on estimating the number of common neighbors for each p -tuple of $U(G)$. These per-tuple common neighbor estimators, along with their higher moments, are aggregated to form an unbiased estimate of the (p, q) -biclique count.

Algorithm 3: The MRCN algorithm ($p = 2$)

Input: G : a bipartite graph; ϵ : total privacy budget;
Output: Private estimate of the number of $(2, q)$ -bicliques

- 1 Split ϵ into ϵ_0, ϵ_1 , and ϵ_2 ;
- // **Phase 0: Degree Estimation**
- 2 **foreach** $u \in V(G)$ **do**
- 3 $\tilde{d}(u) \leftarrow \deg(u, G) + \text{Lap}(1/\epsilon_0)$;
- // **Phase 1: Noisy Graph Construction**
- 4 $p_r \leftarrow \frac{1}{1+e^{\epsilon_1}}$;
- 5 **foreach** $i \in U(G)$ **do**
- 6 **foreach** $j \in L(G)$ **do**
- 7 $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } p_r \\ \mathcal{A}[i, j], & \text{w.p. } 1 - p_r \end{cases}$;
- // **Phase 2: Local Common Neighbor Estimation**
- 8 $\gamma \leftarrow \frac{1-p_r}{1-2p_r}$;
- 9 **foreach** $u, w \in U(G'), u < w$ **do**
- 10 $S_1 \leftarrow 0; S_2 \leftarrow 0$;
- 11 **foreach** $v \in N(u, G)$ **do**
- 12 **if** $\mathcal{A}'[v, w] = 1$ **then**
- 13 $S_1 \leftarrow S_1 + 1$;
- 14 **else**
- 15 $S_2 \leftarrow S_2 + 1$;
- 16 $f_u \leftarrow \gamma \cdot S_1 - \frac{p_r}{1-2p_r} \cdot S_2 + \text{Lap}\left(\frac{1-p_r}{(1-2p_r)\epsilon_2}\right)$;
- // **Phase 3: Moment Correction and Biclique Estimation**
- 17 $\hat{\mathbb{K}}_{2,q} \leftarrow 0$;
- 18 **foreach** $u, w \in U(G'), u < w$ **do**
- 19 $\hat{\text{Var}}(f_u) \leftarrow \frac{p_r(1-p_r)(\tilde{d}(u))}{(1-2p_r)^2} + \frac{2(1-p_r)^2}{(1-2p_r)^2\epsilon_2^2}$;
- 20 $\binom{\omega(u, w)}{q} \leftarrow$ unbiased estimate via moment-based correction;
- 21 $\hat{\mathbb{K}}_{2,q} \leftarrow \hat{\mathbb{K}}_{2,q} + \binom{\omega(u, w)}{q}$;
- 22 **return** $\hat{\mathbb{K}}_{2,q}$;

$$\phi(v_1, u_2)\phi(v_2, u_2) + \phi(v_1, u_3)\phi(v_2, u_3) + \phi(v_1, u_4)\phi(v_2, u_4) = f_1$$

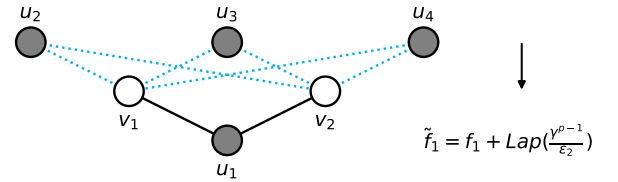


Figure 3: Illustration of the estimator for $\omega(u_1, u_2, u_3, u_4)$ from the neighborhood of u_1 .

Estimating common neighbors for each p -tuple in $U(G)$. Assume that the degree estimates and the noisy graph G_{ϵ_1} are constructed in previous rounds. Consider each p -tuple $X = \{u_1, u_2, \dots, u_p\}$ in $U(G)$. Without loss of generality, let u_1 be the vertex in X with the smallest estimated degree. We obtain the unbiased estimator for $\omega(X)$ in the following theorem.

THEOREM 3. Let $X = \{u_1, u_2, \dots, u_p\} \subseteq U(G)$ be a p -tuple of vertices. Assume that $\phi(v, u_j)$ is an unbiased estimator of the true adjacency $A[v, u_j] \in \{0, 1\}$, i.e., $\mathbb{E}[\phi(v, u_j)] = A[v, u_j]$. Then, the

estimator

$$f_1 = \sum_{v \in N(u_1)} \prod_{j>1} \phi(v, u_j)$$

is an unbiased estimator of the number of common neighbors of $u_1, u_2, \dots, u_p$, i.e., $\omega(X)$.

PROOF. For each $v \in N(u_1)$, define $S_v = \prod_{j>1} A[v, u_j]$. Thus, the number of common neighbors among X is $\omega(X) = \sum_{v \in N(u_1)} S_v$. Consider the estimator $f_1 = \sum_{v \in N(u_1)} \prod_{j>1} \phi(v, u_j)$. By independence and unbiasedness of each $\phi(v, u_j)$,

$$\mathbb{E}[f_1] = \sum_{v \in N(u_1)} \prod_{j>1} \mathbb{E}[\phi(v, u_j)] = \sum_{v \in N(u_1)} \prod_{j>1} A[v, u_j] = \omega(X).$$

This completes the proof. $\square$

To release f_1 under edge LDP, we apply the Laplace mechanism. Let $\gamma = \frac{1-p_r}{1-2p_r}$. Since each $\phi(v, u_j)$ is bounded by γ , the global sensitivity of f_1 is bounded by $\Delta(f_1) \leq \gamma^{p-1}$. Therefore, an unbiased differentially private estimator for $\omega(X)$ is

$$\hat{f}_1 = \sum_{v \in N(u_1)} \prod_{j>1} \phi(v, u_j) + \text{Lap}\left(\frac{\gamma^{p-1}}{\epsilon_2}\right). \quad (2)$$

For example, Fig. 3 illustrates the computation of the f_1 estimator for vertex u_1 in a bipartite graph, where $X = \{u_1, \dots, u_4\}$. Specifically, for u_1 , the neighborhood is $N(u_1) = \{v_1, v_2\}$. This figure highlights the use of edge estimators $\phi(v_j, u_k)$ from neighboring vertices v_j , combined with Laplace noise $\text{Lap}(\frac{\gamma^{p-1}}{\epsilon_2})$ to ensure edge LDP. This generalizes the per-vertex-pair common neighbor estimator from Section 4.1. To estimate the number of (p, q) -biclques containing X , we must properly estimate $\binom{\omega(X)}{q}$. Directly substituting $\hat{f}_1$ into $\binom{\hat{f}_1}{q}$ introduces significant bias, since expectation is not preserved under nonlinear transformations. To correct this, as in the case $p = 2$, we estimate $\text{Var}(\hat{f}_1)$ and apply a moment-based bias adjustment. Define $\theta = \frac{p(1-p_r)}{1-2p_r}$ and $\gamma = \frac{1-p_r}{1-2p_r}$. We carefully analyze the variance of $\hat{f}_1$ as follows.

$$\begin{aligned} \text{Var}(\hat{f}_1) &= \text{Var}\left(\sum_{v \in N(u_1)} \prod_{j>1} \phi(v, u_j)\right) + \text{Var}\left(\text{Lap}\left(\frac{\gamma^{p-1}}{\epsilon_2}\right)\right) \\ &= \sum_{v \in N(u_1)} \text{Var}\left(\prod_{j>1} \phi(v, u_j)\right) + 2 \cdot \frac{\gamma^{2p-2}}{\epsilon_2^2} \end{aligned}$$

We investigate the first term on the right hand side:

$$\begin{aligned} &\sum_{v \in N(u_1)} \text{Var}\left(\prod_{j>1} \phi(v, u_j)\right) \\ &= \sum_{v \in N(u_1)} \left(\prod_{j>1} \mathbb{E}[\phi(v, u_j)^2] - \prod_{j>1} (\mathbb{E}[\phi(v, u_j)])^2 \right) \\ &= \sum_{v \in N(u_1)} \left(\prod_{j>1} (\mathcal{A}[v, u_j]^2 + \theta) - \prod_{j>1} \mathcal{A}[v, u_j]^2 \right) \\ &= \sum_{\substack{Y \subseteq X \\ u_1 \notin Y}} \theta^{p-1-|Y|} \sum_{v \in N(u_1)} \prod_{u_j \in Y} \mathcal{A}[v, u_j]^2 \end{aligned}$$

Note that $\sum_{v \in N(u_1)} \prod_{u_j \in Y} \mathcal{A}[u_j, v]$ corresponds to the number of common neighbors among the vertex set $Y \cup \{u_1\}$. Under edge LDP, this can be unbiasedly estimated by

$$\sum_{v \in N(u_1)} \prod_{u_j \in Y} \phi(v, u_j) + \text{Lap}\left(\frac{\gamma^{|Y|}}{\epsilon_2}\right).$$

Therefore, $\text{Var}(\hat{f}_1)$ can be estimated via:

$$\sum_{\substack{Y \subseteq X \\ u_1 \notin Y}} \theta^{p-1-|Y|} \left(\sum_{v \in N(u_1)} \prod_{u_j \in Y} \phi(v, u_j) + \text{Lap}\left(\frac{\gamma^{|Y|}}{\epsilon_2}\right) \right) + 2 \frac{\gamma^{2p-2}}{\epsilon_2^2}. \quad (3)$$

Using the unbiased estimator $\hat{f}_1$ computed from the neighborhood of u_1 and its estimated variance $\text{Var}(\hat{f}_1)$, we apply the same moment-correction technique as in the case $p = 2$ to approximate higher moments of $\hat{f}_1$ and estimate the number of (p, q) -biclques containing X .

The MRCN algorithm. Algorithm 4 presents the MRCN algorithm based on the discussion above. Phase 0 estimates vertex degrees, and Phase 1 constructs a noisy graph via randomized response. In Phase 2, MRCN iterates over each p -tuple in the upper layer. For each tuple X , it selects the vertex with the smallest estimated degree and traverses its neighbors to estimate the number of common neighbors $\omega(X)$. MRCN computes the variance of the estimator and applies a moment-based correction to obtain unbiased estimates of higher-order moments. Using these, MRCN constructs an unbiased estimator for $\binom{\omega(X)}{q}$ and aggregates the results across all p -tuples to produce the final unbiased estimate of the (p, q) -biclque count.

Computational time complexity. Randomized responses in MRCN takes $O\left(n_1 n_2 \cdot \frac{1}{1+e^{\epsilon_1}}\right)$. Instead of enumerating all (p, q) -motifs as in OneR, MRCN processes each p -tuple by selecting a source vertex, exploring its neighbors, and computing the product of $\phi(v, x_j)$ over the remaining vertices in the tuple. This costs $O(d_{\max}^1 \cdot p)$ per p -tuple, with $\binom{n_1}{p}$ tuples overall ($d_{\max}^1$ is the maximum degree on $U(G)$). Hence, the total time complexity is $O\left(\frac{n_1 n_2 + \binom{n_1}{p} d_{\max}^1}{1+e^{\epsilon_1}}\right)$.

Per-vertex Communication costs. The dominating communication cost of MRCN occurs when downloading the noisy graph. The size of the noisy graph is approximately $m(1-p_r) + (n_1 n_2 - m)p_r = O(\frac{n_1 n_2}{1+e^{\epsilon}})$ in expected value. Since each noisy edge needs a

Algorithm 4: The MRCN algorithm

Input: G : a bipartite graph; ϵ : total privacy budget; p, q : biclique parameters

Output: Private estimate of the number of (p, q) -bicliques

```

1 Split  $\epsilon$  into  $\epsilon_0, \epsilon_1$ , and  $\epsilon_2$ ;
  // Phase 0: Degree Estimation
2 foreach  $u \in V(G)$  do
3    $\hat{d}(u) \leftarrow \deg(u, G) + \text{Lap}(1/\epsilon_0)$ ;
  // Phase 1: Noisy Graph Construction
4  $p_r \leftarrow \frac{1}{1+e^{\epsilon_1}}, \gamma \leftarrow \frac{1-p_r}{1-2p_r}$ ;
5 foreach  $i \in U(G)$  do
6   foreach  $j \in L(G)$  do
7      $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{with probability } p_r \\ \mathcal{A}[i, j], & \text{with probability } 1 - p_r \end{cases}$ ;
  // Phase 2: Common Neighbor Estimation
8 Initialize  $\hat{\mathbb{K}}_{p,q} \leftarrow 0$ ;
9 foreach  $p$ -tuple  $X = (x_1, x_2, \dots, x_p)$  from  $U(G)$  do
10   $S \leftarrow 0$ ;
11   $x_1 \leftarrow$  the vertex with the smallest estimated degree in  $x_i$ ;
12  foreach  $v \in N(x_1, G)$  do
13     $prod \leftarrow 1$ ;
14    for  $j = 2$  to  $p$  do
15       $\phi \leftarrow \frac{\mathcal{A}'[v, x_j] - p_r}{1-2p_r}$ ;
16       $prod \leftarrow prod \times \phi$ ;
17     $S \leftarrow S + prod$ ;
18   $\hat{f}_1 \leftarrow S + \text{Lap}(\frac{\gamma^p}{\epsilon_2})$ ;
19   $\text{Var}(\hat{f}_1) \leftarrow$  estimate variance of  $\hat{f}_1$  from Equation 3;
20   $(\omega_q^{(X)}) \leftarrow$  unbiased estimate via moment-based correction;
21   $\hat{\mathbb{K}}_{p,q} \leftarrow \hat{\mathbb{K}}_{p,q} + (\omega_q^{(X)})$ ;
22 return  $\hat{\mathbb{K}}_{p,q}$ ;
```

cost of $O(\log(n_1 n_2))$, the overall per-vertex communication cost is $O(\frac{n_1 n_2 \log(n_1 n_2)}{1+e^\epsilon})$.

4.3 Optimizations

In this section, we present several optimizations to enhance the data utility and efficiency of MRCN. Specifically, we introduce the *double-noisy graph* and *multi-center* optimizations to improve accuracy, and propose *layer-based optimization* and *vertex sampling* to boost scalability and runtime efficiency.

Multi-center optimization. To reduce the variance of biclique estimators under edge LDP, we propose a *multi-center optimization* that aggregates estimators from all vertices in each p -tuple, rather than relying on a single center vertex as in the basic MRCN algorithm. While the single-center approach yields an unbiased estimate, constructing estimators $f_1, f_2, \dots, f_p$ from each vertex in the p -tuple and averaging them improves both accuracy and robustness. Based on Equation 2, for each vertex v_i , we can compute the estimator f_i , as given by

$$f_i = \sum_{u \in N(v_i)} \prod_{j \neq i} \phi(u, v_j) + \text{Lap}\left(\frac{\gamma^{p-1}}{\epsilon_2}\right),$$

where $\phi(u, v_j)$ is the unbiased estimator $\mathcal{A}[u, v_j]$, and $\gamma = \frac{1-p_r}{1-2p_r}$. We then average these: $f_{\text{avg}} = \frac{1}{p} \sum_{i=1}^p f_i$, which remains unbiased due to the linearity of expectation.

To accurately estimate the variance of f_{avg} , we explicitly account for the covariance between each pair (f_i, f_j) . Let $\text{Cov}(f_i, f_j)$ denote the covariance between f_i and f_j , which arises due to overlapping edge estimators. We model these using intermediate cross terms (e.g., f_{ij} and f_{ji}), computed as partial sums over $\phi(u, v_j)$ from v_i 's neighborhood and vice versa. The variance of f_{avg} is:

$$\text{Var}(f_{\text{avg}}) = \frac{1}{p^2} \left(\sum_{i=1}^p \text{Var}(f_i) + 2 \sum_{i < j} \text{Cov}(f_i, f_j) \right),$$

This is used in the moment-correction technique for estimating the per-tuple biclique counts. Here, we illustrate how to compute $\text{Var}(f_{\text{avg}})$ when $p = 2$ and $p = 3$.

(1) When $p = 2$: For the 2-tuple (u_1, u_2) , we construct estimators $\hat{f}_1$ and $\hat{f}_2$ based on the neighborhoods of u_1 and u_2 , respectively. Specifically, $\hat{f}_1$ depends only on entries $\mathcal{A}[v_i, u_2]$ where $v_i \in N(u_1)$, and $\hat{f}_2$ depends only on entries $\mathcal{A}[v_i, u_1]$ where $v_i \in N(u_2)$. Since these entries are disjoint in the noisy graph, $\hat{f}_1$ and $\hat{f}_2$ are independent, implying $\text{Cov}(\hat{f}_1, \hat{f}_2) = 0$. Thus, the variance of the average estimator can be computed as $\text{Var}(f_{\text{avg}}) = \frac{1}{4} (\text{Var}(\hat{f}_1) + \text{Var}(\hat{f}_2))$.

(2) When $p = 3$: For the triple (u_1, u_2, u_3) , we compute $\hat{f}_i$ ($i=1,2,3$) via Equation 2. In this case, $\hat{f}_i$ are no longer independent due to some common terms. Specifically, the covariance between $\hat{f}_1$ and $\hat{f}_2$ arises from their shared neighbors in $N(u_1) \cap N(u_2)$:

$$\text{Cov}(\hat{f}_1, \hat{f}_2) = \text{Var}(\phi) \cdot |N(u_1) \cap N(u_2)|,$$

where $\text{Var}(\phi) = \frac{p_r(1-p_r)}{(1-2p_r)^2}$. Thus, these covariances require us to estimate the pair-wise common neighbors.

$$f_{12} = \sum_{v \in N(u_1)} \phi(v, u_2) + \text{Lap}\left(\frac{\gamma}{\epsilon_2}\right), \quad f_{21} = \sum_{v \in N(u_2)} \phi(v, u_1) + \text{Lap}\left(\frac{\gamma}{\epsilon_2}\right),$$

$$\text{Cov}(\hat{f}_1, \hat{f}_2) \approx \text{Var}(\phi) \cdot \frac{f_{12} + f_{21}}{2}.$$

The final averaged estimator and its estimated variance are:

$$\hat{f}_{\text{avg}} = \frac{\hat{f}_1 + \hat{f}_2 + \hat{f}_3}{3}, \quad \text{Var}(\hat{f}_{\text{avg}}) = \frac{1}{9} \left(\sum_{i=1}^3 \text{Var}(\hat{f}_i) + 2 \sum_{i < j} \text{Cov}(\hat{f}_i, \hat{f}_j) \right).$$

Refined noisy graph construction. Edge LDP algorithms often apply randomized responses to either the upper or lower triangle of the adjacency matrix to avoid conflicts in the noisy graph. However, this halves the usable data and limits estimation accuracy. We propose the *Refined noisy graph construction*, which applies randomized responses to each vertex in G and uses the upper and the lower triangle of the noisy adjacency matrix to build two independent noisy graphs. This still complies with edge LDP by parallel composition and post-processing properties [25]. This allows us to improve the unbiased edge estimator $\phi(i, j) = \frac{\mathcal{A}'[i, j] - p_r}{1-2p_r}$ by averaging over the two noisy graphs: $\phi^{\text{avg}}(i, j) = \frac{1}{2} (\phi^{(1)}(i, j) + \phi^{(2)}(i, j))$, which remains unbiased and has half the variance: $\text{Var}(\phi^{\text{avg}}(i, j)) = \frac{1}{2} \text{Var}(\phi(i, j))$. Since many estimators for common neighbors and

(p, q) -bicliques are based on $\phi(i, j)$, this directly improves overall estimation accuracy.

Layer-based optimization and vertex tuple sampling. The MRCN algorithm estimates (p, q) -biclique by enumerating p -tuples in the upper layer $U(G)$. A symmetric approach can be applied using q -tuples from the lower layer $L(G)$. These two directions yield different time complexities: $O\left(\frac{n_1 n_2 + \binom{n_1}{p} d_{\max}^1}{1 + \epsilon^{\epsilon_1}}\right)$ for enumerating p -tuples in $U(G)$, and $O\left(\frac{n_1 n_2 + \binom{n_2}{q} d_{\max}^2}{1 + \epsilon^{\epsilon_1}}\right)$ for enumerating q -tuples in $L(G)$, where $d_{\max}^1$ and $d_{\max}^2$ are the maximum degrees of $U(G)$ and $L(G)$. Intuitively, we can pre-compute the costs of both options and select the cheaper one, which achieves a significant speedup on real-world bipartite graphs with imbalanced layers. Degree estimates, rather than true degrees, guide cost estimation. Note that we use degree estimates from Phase 0 when precomputing the computational costs. To maintain scalability when the number of p -tuples or q tuples is large, we apply uniform sampling to preserve unbiasedness. In experiments, the sample size is fixed at $\tau = 10^6$.

5 EXPERIMENTAL EVALUATION

In this section, we evaluate the effectiveness and efficiency of the proposed (p, q) -biclique counting algorithms under ϵ -edge LDP through experiments.

Table 2: Summary of Datasets

Dataset	Upper	Lower	$ E $	$ U $	$ L $
Unicode (UN)	Country	Language	1.26K	255	615
Ircwiki (LR)	User	Article	18.9K	209	8.74K
filmtrust (LS)	User	Film	35.5K	1.5K	2.07K
rmwiki (RM)	User	Article	57.9K	1.22K	8.09K
Condmatt (AC)	Author	Paper	58.6K	16.7K	22.0K
csbwiki (CS)	User	Article	62.1K	1.33K	8.28K
bag-kos (BK)	Document	Word	353K	3.43K	6.90K
bpywiki (BP)	User	Article	399.7K	1.32K	57.9K
NIPS (NS)	Document	Word	746.3K	1.50K	12.4K
lastfm band (LB)	User	Band	898K	38.7M	16.7M
Discogs (DL)	Label	Style	1.06M	243.8K	0.38K
DiggVotes (DV)	User	Story	3.01M	139.4K	3.55K
MovieLens (MV)	User	Movie	10.0M	69.9K	10.7K

5.1 Experimental Settings

Datasets. We use 13 datasets from KONECT (<http://konect.cc/>). Table 2 shows the statistics of the datasets. $|U|$ and $|L|$ are the number of vertices in the upper and lower layers. $|E|$ is the number of edges in the graph.

Algorithms. We evaluate the following (p, q) -biclique counting algorithms under edge LDP.

- (1) Naive: the algorithm directly returning the (p, q) -biclique count on the noisy graph;
- (2) OneR: the one-round algorithm that computes unbiased estimates by counting (p, q) -vertex motifs in the noisy graph;
- (3) MRCN: a novel multi-round algorithm based on common neighbor estimation;
- (4) MRCN++: MRCN with refined noisy graph construction, multi-center aggregation, and layer-based optimizations;

All algorithms are implemented in C++. The experiments are run

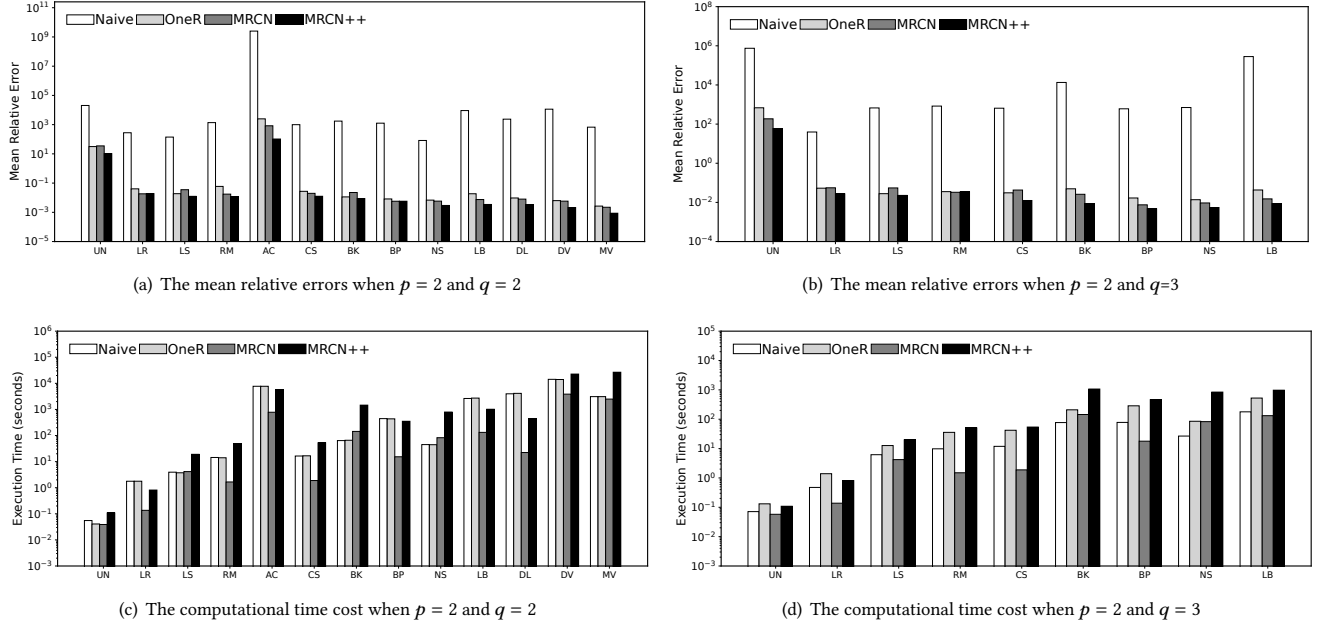
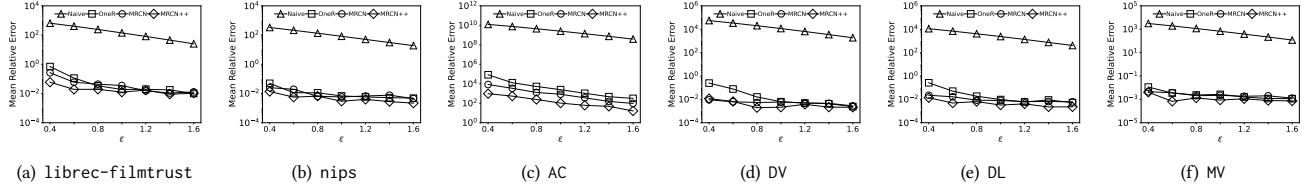
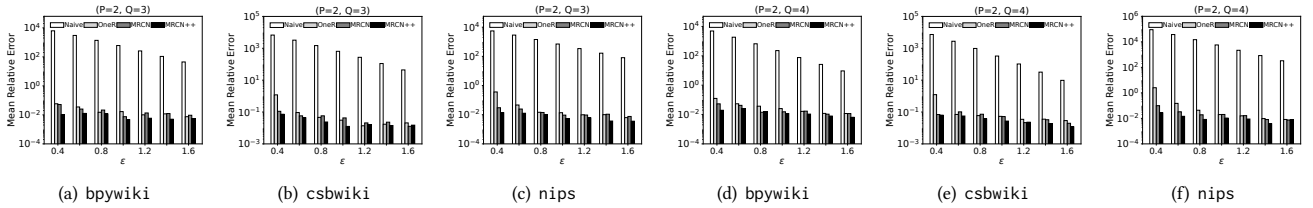
on a Linux server with an Intel Xeon 6342 processor and 512GB of memory.

Parameter settings. By default, the privacy budget ϵ is set to 1. We also allow it to vary from 0.4 to 1.6 with increments of 0.2. For MRCN and MRCN++, we allocate the privacy budget as follows: $\epsilon_0 = 0.05\epsilon$, $\epsilon_1 = 0.6\epsilon$, and $\epsilon_2 = 0.35\epsilon$. When $p > 2$, we apply vertex sampling to improve the efficiency of p -tuple enumeration. Let τ denote the number of p -tuples/ q -tuple sampled, which is set to 10^6 by default.

Evaluate the performance of (p, q) -biclique counting algorithms under edge LDP across datasets. In Fig. 4(a) and Fig. 4(b), we report the performances of Naive, OneR, MRCN, and MRCN++ across different datasets when $\epsilon = 1$, $p = 2$, and $q = 2, 3$. First, we observe that OneR, MRCN, and MRCN++ significantly outperform Naive across all datasets. This is because these algorithms produce unbiased estimates of (p, q) -bicliques, which addresses the large bias issue in Naive. We can also see that on most datasets, MRCN and MRCN++ further outperform OneR. This is because the novel multi-round algorithms MRCN and MRCN++ estimate the number of common neighbors for each p -tuple in the upper layer or q -tuple in the lower layer of G , and leverage robust common neighbor estimators and their higher moments to derive unbiased per-tuple biclique counts. Also, we observe that MRCN++ consistently improves the data utility of MRCN. This is because MRCN++ adopts multi-center optimization and refined noisy graph construction to reduce the variance of the common neighbor estimators as well as the resulting per-tuple biclique estimators.

Evaluate the computational time costs across datasets. In Fig. 4(c) and Fig. 4(d), we report the computational time costs of Naive, OneR, MRCN and MRCN++ when $(p = 2, q = 2)$ and $(p = 2, q = 3)$, $\epsilon = 1$. In general, Naive and OneR incur higher time costs on sparse graphs like AC and LB, due to the combinatorial cost of counting (p, q) -bicliques on the noisy graph. In contrast, MRCN and MRCN++ avoid full motif enumeration and leverage local common neighbor estimations with cost proportional to $\binom{n_1}{p} \cdot d_{\max}^{q-1}$, which is significantly lower when the graph is sparse. We also observe that OneR generally incurs higher time costs than Naive. Although both methods apply randomized response with the full privacy budget, their counting phase differ: Naive counts only the (p, q) -bicliques on the noisy graph, whereas OneR must enumerate all motifs with p upper and q lower vertices. This gap becomes more pronounced when $q = 3$, as the number of possible $(2, 3)$ -vertex motifs significantly exceeds that of $(2, 2)$ -vertex motifs. In addition, we observe that MRCN++ incurs higher running time than MRCN, as expected. This is due to its use of multiple source vertices for common neighbor estimation and the construction of two noisy graphs from the upper and lower triangles of the perturbed adjacency matrix.

Evaluate the effect of the privacy budget ϵ . In Fig. 5, Fig. 6, and Fig. 7, we report the mean relative errors of Naive, OneR, MRCN, and MRCN++ across datasets, as ϵ varies between 0.4 and 1.6. Note that we provide an efficient implementation of the OneR algorithm only for the case $p = 2$, as extending it to general $p > 2$ is challenging due to the intractable number of distinct (p, q) -motif types. Overall, we observe that the data utility of all algorithms improves as the privacy budget increases. Specifically, we observe that MRCN, MRCN++, and OneR consistently outperform Naive by orders of magnitude

Figure 4: Performance on different datasets ($\epsilon = 1$)Figure 5: Evaluate the effect of ϵ when $p = 2$, $q = 2$.Figure 6: Evaluate the effect of ϵ for $(p = 2, q = 3)$ and $(p = 2, q = 4)$ on 3 datasets.

across various (p, q) settings, as they rely on provably unbiased estimators rather than directly counting bicliques on the noisy graph. As shown in Fig. 5 and Fig. 6, when $p = 2$, MRCN and MRCN++ generally yield lower mean relative errors than OneR, with the advantage becoming more pronounced on sparse graphs. This is because OneR results in high variance because each (p, q) -biclique depends on roughly $p \times q$ independent Bernoulli variables. MRCN and MRCN++ address the issue of OneR by allowing the vertices to download the noisy graph and estimate the number of common neighbors for each vertex pair (u, w) locally, which involves $\deg(u)$ or $\deg(w)$

random variables. In Fig. 7 ($p = 3$), both MRCN and MRCN++ significantly outperform the Naive algorithm. We omit OneR as it requires enumerating all motifs with p upper and q lower vertices, which becomes infeasible on real-world bipartite graphs when $p > 2$.

Evaluate the communication costs of all algorithms. In Fig. 8, we report the per-vertex communication costs (in MB) of each algorithm as ϵ varies. Fig. 8 (a)–(c) show results for $(p = 2, q = 2)$, while Fig. 8 (d)–(f) correspond to $(p = 2, q = 3)$. We observe that Naive and OneR incur nearly identical communication costs. This is

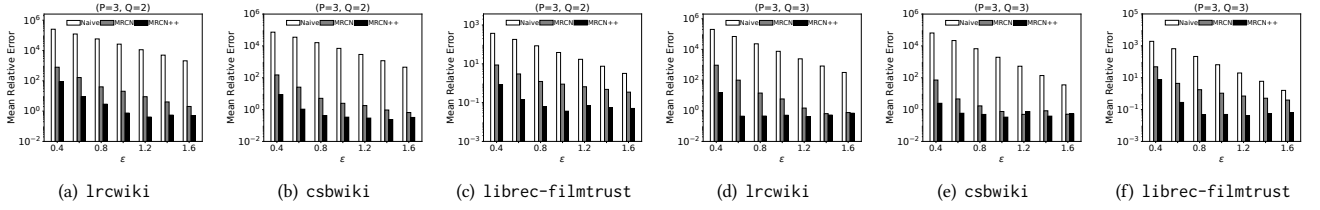


Figure 7: Evaluate the effect of ϵ for $(p=3, q=2)$ and $(p=3, q=3)$ on 3 datasets. $\tau = 10^6$.

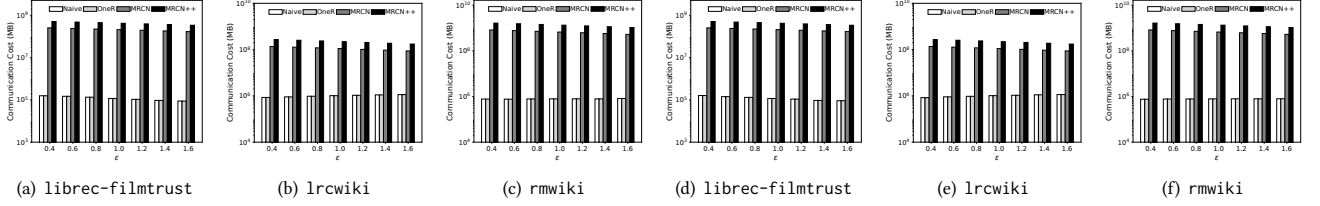


Figure 8: Evaluate the communication costs for $(p=2, q=2)$ and $(p=2, q=3)$.

because both algorithms use the randomized responses with identical flip probabilities. In contrast, MRCN and MRCN++ incur higher communication costs because the multi-round protocol requires: (1) uploading noisy edges to the curator, (2) downloading noisy edges for all vertices, and (3) uploading the common-neighbor estimators to the data curator. MRCN++ incurs the highest communication costs from its refined noisy graph construction and multi-center optimization, sending more noisy edges and local estimators.

6 RELATED WORK

We review the related works on (p, q) -biclique counting on bipartite graphs and motif counting under edge LDP.

(p, q) -biclique counting. Yang et al. [39] first systematically study (p, q) -biclique enumeration and counting and propose the BCList and BCList++ algorithms. BCList adopts a branch-and-bound framework with pruning but has exponential complexity in $p+q$. BCList++ reduces this to exponential in either p or q by anchoring the search to a single partition, while incorporating a cost model, memory reuse, and parallelism for scalability. To avoid full enumeration, Ye et al. [41] propose EPivoter, which encodes bicliques via edge-pivoting, and two sampling-based methods, ZigZag and ZigZag++, using h -zigzag dynamic programming. To further accelerate counting, Qiu et al. [27] develop GBC, which uses hierarchical truncated bitmaps for efficient set intersections, a hybrid DFS-BFS search for better GPU parallelism, and a composite load-balancing scheme for workload distribution. The special case of (p, q) -biclique with $p = q = 2$ is known as a butterfly, and butterfly counting has been widely studied [19, 28–30, 33–36, 38, 44].

Motif counting under edge-local differential privacy. Motif counting under CDP has been widely studied for various structures such as triangles, k -stars, and k -triangles [3, 18, 22, 42]. However, they assume full access to the graph and are not applicable under edge LDP [6, 13–15, 21, 32]. Edge LDP [25] has emerged as a widely accepted privacy model for graph mining. In this setting, [13]

proposes one-round and two-round triangle counting algorithms, further improved by [14] for better communication efficiency and accuracy. [6] offers a detailed analysis of these techniques, while [11] introduces a vertex-centric algorithm that enhances utility. Localized variants [21, 32] allow each vertex to report its 2-hop neighborhood but require a stronger LDP model. Beyond triangle motifs, 4-cycle counting has been studied under the shuffle model [15]. On bipartite graphs, He et al. [9] propose one-round and multi-round butterfly counting algorithms under edge LDP. The multi-round method extends triangle counting by estimating per-vertex butterfly counts, but suffers from high Laplace noise due to motif complexity. Since butterflies are $(2, 2)$ -bicliques and general (p, q) -bicliques are more complex, we design a new multi-round algorithm that avoids per-vertex counts to improve accuracy under edge LDP. Also, [10] studies common neighbor counting on bipartite graphs. For common neighbor counting [10], we build on their multi-round estimator, adapting it in MRCN to use shared privacy budget and weighting parameters. A recent work [31] proposes k -star LDP for (p, q) -biclique counting. We do not compare to it because (1) it relies on k -star adjacency lists, and (2) its edge LDP variant transmits local estimates without proper Laplace calibration and violates edge LDP. (1) [31] focuses on introducing k -star LDP, which is based on k -star adjacency lists rather than neighbor lists in edge LDP. (2) [31] only provided a baseline edge LDP solution without Laplace noise, which violates edge LDP.

7 CONCLUSION

In this paper, we tackle (p, q) -biclique counting in bipartite graphs under edge LDP. The Naive algorithm directly counts (p, q) -bicliques on noisy graphs and results in significant bias. To address this, we introduce the OneR algorithm, which employs motif transformation probabilities to provide unbiased estimates. For enhanced generalizability and data utility, we introduce the MRCN algorithm, which refines common neighbor estimates for p -tuples in the upper layer

or q -tuples in the lower layer, utilizing higher moments to achieve superior accuracy. Additionally, we propose refined noisy graph construction and multiple-center optimization to enhance accuracy further, as well as layer-based optimization and vertex sampling to improve efficiency. Extensive experiments on real-world datasets validate the effectiveness and efficiency of our proposed algorithms.

REFERENCES

- [1] Xiaoshuang Chen, Kai Wang, Xuemin Lin, Wenjie Zhang, Lu Qin, and Ying Zhang. 2021. Efficiently answering reachability and path queries on temporal bipartite graphs. *Proceedings of the VLDB Endowment* (2021).
- [2] Wei-Yen Day, Ninghui Li, and Min Lyu. 2016. Publishing graph degree distribution with node differential privacy. In *Proceedings of the 2016 International Conference on Management of Data*. 123–138.
- [3] Xiaofeng Ding, Shujun Sheng, Huajian Zhou, Xiaodong Zhang, Zhifeng Bao, Pan Zhou, and Hai Jin. 2021. Differentially private triangle counting in large graphs. *IEEE Transactions on Knowledge and Data Engineering* 34, 11 (2021), 5278–5292.
- [4] Xiaofeng Ding, Xiaodong Zhang, Zhifeng Bao, and Hai Jin. 2018. Privacy-preserving triangle counting in large graphs. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 1283–1292.
- [5] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.
- [6] Talya Eden, Quanquan C Liu, Sofya Raskhodnikova, and Adam Smith. 2023. Triangle Counting with Local Edge Differential Privacy. *arXiv preprint arXiv:2305.02263* (2023).
- [7] Michael Hay, Chao Li, Gerome Miklau, and David Jensen. 2009. Accurate estimation of the degree distribution of private networks. In *2009 Ninth IEEE International Conference on Data Mining*. IEEE, 169–178.
- [8] Michael Hay, Vibhor Rastogi, Gerome Miklau, and Dan Suciu. 2009. Boosting the accuracy of differentially-private histograms through consistency. *arXiv preprint arXiv:0904.0942* (2009).
- [9] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Wei Ni, and Ying Zhang. 2024. Butterfly Counting over Bipartite Graphs with Local Differential Privacy. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2351–2364.
- [10] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2024. Common Neighborhood Estimation over Bipartite Graphs under Local Differential Privacy. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.
- [11] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Wei Ni. 2025. Robust Privacy-Preserving Triangle Counting under Edge Local Differential Privacy. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [12] Zan Huang, Xin Li, and Hsinchun Chen. 2005. Link prediction approach to collaborative filtering. In *Proceedings of the 5th ACM/IEEE-CS joint conference on Digital libraries*. 141–142.
- [13] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally Differentially Private Analysis of Graph Statistics. In *USENIX Security Symposium*. 983–1000.
- [14] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Communication-Efficient Triangle Counting under Local Differential Privacy. In *31st USENIX Security Symposium (USENIX Security 22)*. 537–554.
- [15] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Differentially Private Triangle and 4-Cycle Counting in the Shuffle Model. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1505–1519.
- [16] Leon Isserlis. 1918. On a formula for the product-moment coefficient of any order of a normal frequency distribution in any number of variables. *Biometrika* 12, 1/2 (1918), 134–139.
- [17] Honglu Jiang, Jian Pei, Dongxiao Yu, Jiguo Yu, Bei Gong, and Xiuzhen Cheng. 2021. Applications of differential privacy in social network analysis: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 1 (2021), 108–127.
- [18] Vishesh Karwa, Sofya Raskhodnikova, Adam Smith, and Grigory Yaroslavtsev. 2011. Private analysis of graph structure. *Proceedings of the VLDB Endowment* 4, 11 (2011), 1146–1157.
- [19] Rundong Li, Pinghui Wang, Peng Jia, Xiangliang Zhang, Junzhou Zhao, Jing Tao, Ye Yuan, and Xiaohong Guan. 2021. Approximately counting butterflies in large bipartite graph streams. *IEEE Transactions on Knowledge and Data Engineering* 34, 12 (2021), 5621–5635.
- [20] Zhao Li, Xin Shen, Yuhang Jiao, Xuming Pan, Pengcheng Zou, Xianling Meng, Chengwei Yao, and Jiajun Bu. 2020. Hierarchical bipartite graph neural networks: Towards large-scale e-commerce applications. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1677–1688.
- [21] Yuhan Liu, Suyun Zhao, Yixuan Liu, Dan Zhao, Hong Chen, and Cuiping Li. 2022. Collecting triangle counts with edge relationship local differential privacy. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2008–2020.
- [22] Tianzi Lv, Huanzhou Li, Zhangguo Tang, Fangzhou Fu, Jian Cao, and Jian Zhang. 2021. Publishing Triangle Counting Histogram in Social Networks Based on Differential Privacy. *Security and Communication Networks* 2021 (2021), 1–16.
- [23] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2007. Smooth sensitivity and sampling in private data analysis. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*. 75–84.
- [24] Clare M O'Connor, Jill U Adams, and Jennifer Fairman. 2010. Essentials of cell biology. *Cambridge, MA: NPG Education* 1 (2010), 54.
- [25] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating synthetic decentralized social graphs with local differential privacy. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 425–438.
- [26] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating synthetic decentralized social graphs with local differential privacy. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 425–438.
- [27] Linshan Qiu, Zhonggen Li, Xiangyu Ke, Lu Chen, and Yunjun Gao. 2024. Accelerating Biclique Counting on GPU. *arXiv preprint arXiv:2403.07858* (2024).
- [28] Seyed-Vahid Sane'i-Mehri, Ahmet Erdem Sariyuce, and Srikanta Tirathapura. 2018. Butterfly counting in bipartite networks. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2150–2159.
- [29] Aida Sheshbolouki and M Tamer Özsu. 2022. sGrapp: Butterfly approximation in streaming graphs. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 16, 4 (2022), 1–43.
- [30] Jessica Shi and Julian Shun. 2022. Parallel algorithms for butterfly computations. In *Massive Graph Analytics*. Chapman and Hall/CRC, 287–330.
- [31] Henan Sun, Zhengyu Wu, Rong-Hua Li, Guoren Wang, and Zening Li. 2024. K-stars LDP: A Novel Framework for (p, q)-clique Enumeration under Local Differential Privacy. *arXiv preprint arXiv:2403.01788* (2024).
- [32] Haipai Sun, Xiaokui Xiao, Issa Khalil, Yin Yang, Zhan Qin, Hui Wang, and Ting Yu. 2019. Analyzing subgraph statistics from extended local views with decentralized differential privacy. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 703–717.
- [33] Jia Wang, Ada Wai-Chee Fu, and James Cheng. 2014. Rectangle counting in large bipartite graphs. In *2014 IEEE International Congress on Big Data*. IEEE, 17–24.
- [34] Kai Wang, Xuemin Lin, Lu Qin, Wenjie Zhang, and Ying Zhang. 2019. Vertex Priority Based Butterfly Counting for Large-Scale Bipartite Networks. *Proceedings of the VLDB Endowment* 12, 10 (2019), 1139–1152.
- [35] Kai Wang, Xuemin Lin, Lu Qin, Wenjie Zhang, and Ying Zhang. 2020. Efficient bitruss decomposition for large-scale bipartite graphs. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 661–672.
- [36] Zhibin Wang, Longbin Lai, Yixue Liu, Bing Shui, Chen Tian, and Sheng Zhong. 2023. I/O-Efficient Butterfly Counting at Scale. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.
- [37] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *J. Amer. Statist. Assoc.* 60, 309 (1965), 63–69.
- [38] Qingyu Xu, Feng Zhang, Zhiming Yao, Lv Lu, Xiaoyong Du, Dong Deng, and Bing-sheng He. 2022. Efficient load-balanced butterfly counting on GPU. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2450–2462.
- [39] Jianye Yang, Yun Peng, and Wenjie Zhang. 2021. (p, q)-biclique counting and enumeration for large sparse bipartite graphs. *Proceedings of the VLDB Endowment* 15, 2 (2021), 141–153.
- [40] Qingqing Ye, Haibo Hu, Man Ho Au, Xiaofeng Meng, and Xiaokui Xiao. 2020. LF-GDPR: A framework for estimating graph metrics with local differential privacy. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4905–4920.
- [41] Xiaowei Ye, Rong-Hua Li, Qiangqiang Dai, Hongchao Qin, and Guoren Wang. 2023. Efficient Biclique Counting in Large Bipartite Graphs. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
- [42] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2015. Private release of graph statistics using ladder functions. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 731–745.
- [43] Yuxuan Zhang, Jianghong Wei, Xiaojian Zhang, Xuexian Hu, and Wenfen Liu. 2018. A two-phase algorithm for generating synthetic graph under local differential privacy. In *Proceedings of the 8th International Conference on Communication and Network Security*. 84–89.
- [44] Alexander Zhou, Yue Wang, and Lei Chen. 2021. Butterfly counting on uncertain bipartite graphs. *Proceedings of the VLDB Endowment* 15, 2 (2021), 211–223.